

— MINI-CURSO • 2 DIAS • 1H POR AULA

Introdução ao Desenvolvimento **Android**

Dia 1 — montar a tela. **Dia 2** — dar vida a ela.
No fim, construímos juntos o **Hogwarts Quiz**.

Kotlin

Jetpack Compose

Android Studio

O que vamos construir

- ◇ Aparece um **personagem** — você adivinha a **casa de Hogwarts**.
- ◇ **5 rodadas**, cada uma com um personagem vindo da internet.
- ◇ No fim, uma **tela de resultado** com o placar.
- ◇ Só **duas telas** — o suficiente pra ver tudo funcionando junto.



Como o curso funciona

DIA 1 · A TELA

Montar a interface

Como o Android funciona · Kotlin para ler código · o que é o Compose · UI declarativa · **@Composable** e os blocos · **Modifier** · tema e cores.

DIA 2 · A LÓGICA

Dar vida à tela

Estado e recomposição · reagir a cliques · buscar dados da internet · navegar entre telas · montar o quiz inteiro.

DIA 1

Montando a tela

o que é o Android

o que é um app

Kotlin essencial

UI declarativa

@Composable

blocos: Column, Row, Box...

Modifier

tema & cores

O que é o Android

Um sistema operacional sobre um kernel **Linux**. Cada app roda **isolado**, no seu próprio espaço. A gente programa na **camada de cima**, com Kotlin.

Seu app — Hogwarts Quiz

Kotlin + Compose

APIs do Android

telas, sensores, rede

Bibliotecas do sistema

gráficos, mídia

Kernel Linux

memória, energia, drivers

O que é um app Android

ACTIVITY

A porta de entrada

Uma "tela" inicial. O app começa numa Activity e desenha toda a interface dentro dela.

ANDROID STUDIO

Onde a gente trabalha

O programa onde escrevemos, rodamos e testamos o app — com um emulador de celular embutido.

GRADLE

O montador

Junta o código, baixa as bibliotecas (Compose, etc.) e gera o app instalável.

Kotlin, em um slide

A linguagem do Android hoje. Você não precisa dominar — só **reconhecer o básico** pra acompanhar.

- ◇ **val** não muda · **var** muda
- ◇ **fun** declara uma função
- ◇ `String?` pode ser nulo · `?:` dá um valor padrão
- ◇ `$nome` insere um valor dentro do texto

exemplo

```
val nome = "Harry"           // texto, não muda
var pontos = 0                // número, pode mudar

fun saudacao(n: String) = "Olá, $n"

val casa: String? = null
println(casa ?: "Sem casa ainda")
```

— DIA 1 · O MODELO MENTAL

Jetpack Compose: UI declarativa

O JEITO ANTIGO

Você monta a tela em arquivos separados e **atualiza cada pedaço na mão** quando algo muda. Fácil esquecer um caso.

COM O COMPOSE

Você **descreve** como a tela deve ser para um **estado**. Mudou o estado, o Compose redesenha o que for preciso.

`tela = f( estado )`

A função `@Composable`

No Compose, a tela é feita de **funções**. Uma função `@Composable` desenha um pedaço da interface — e pode usar outras dentro dela.

Text — texto

Column — empilha

Row — lado a lado

Box — sobrepõe

Button — botão

Image — imagem

Spacer — espaço

exemplo

```
@Composable
fun Saudacao() {
    Column {
        Text("Bem-vindo a Hogwarts")
        Button(onClick = { }) {
            Text("Começar")
        }
    }
}
```

Modifier

Todo bloco aceita um **Modifier**: tamanho, espaçamento, cor de fundo, borda, clique.
É **encadeado** — e a **ordem importa**.

exemplo

```
Text(  
  "Grifinória",  
  modifier = Modifier  
    .fillMaxWidth()      // ocupa a largura  
    .background(Color.Red) // pinta o fundo  
    .padding(16.dp)       // respiro por dentro  
)
```

Trocar a ordem de `background` e `padding` muda o resultado na tela.

— DIA 1 · IDENTIDADE VISUAL

Tema, cores e @Preview

TEMA CENTRAL

Um lugar só

Cores, tipografia e formas ficam num tema. A tela inteira puxa dali — muda num ponto, muda em tudo.

@PREVIEW

Ver na hora

Desenha a função direto no Android Studio, sem abrir o app no emulador.

ITERAÇÃO

Ajuste rápido

Mexeu no código, o preview atualiza. Dá pra afinar o visual em segundos.

— FIM DO DIA 1

Agora você consegue montar **qualquer tela** estática.

As telas do quiz e de resultado já existem no papel — só não fazem nada ainda. É o que resolvemos amanhã.

DIA 2

Dando **vida** à tela

estado & recomposição

reagir a cliques

dados da internet

API REST · JSON

navegação entre telas

o quiz inteiro

Estado & recomposição

Estado é a informação que muda enquanto o app roda: a rodada atual, o placar, o personagem da vez.

- ◇ **remember { mutableStateOf(...) }** guarda o valor entre redesenhos.
- ◇ Mudou o estado → o Compose **recompõe** só o que usa aquele valor.

exemplo

```
var cliques by remember {  
    mutableStateOf(0)  
}  
  
Button(onClick = { cliques++ }) {  
    Text("Cliques: $cliques")  
}
```

— DIA 2 · INTERAÇÃO

Reagir a um clique



No quiz: tocar numa casa → confere se acertou → soma no placar → vai para a próxima rodada.

Buscar dados da internet

O personagem de cada rodada vem de uma **API pública** — um endereço na web que devolve dados.

API REST

A fonte

Um endereço que responde com os dados do personagem em formato **JSON** (texto estruturado).

RETROFIT + GSON

A ponte

Fazem a chamada e transformam o JSON num **objeto Kotlin** que a tela consegue usar.

COIL

A imagem

Baixa e mostra a **foto** do personagem a partir do link, com cache.

Isso demora um tico → roda em **segundo plano** pra não travar a tela.

Navegar entre telas

- ◇ Cada tela tem um **nome** (rota): "quiz", "resultado".
- ◇ Um **NavHost** lista as telas; pedir pra navegar troca de uma pra outra.
- ◇ Dá pra **passar dados** na troca — por exemplo, o placar final.

Tela do quiz

↓ fim das 5 rodadas

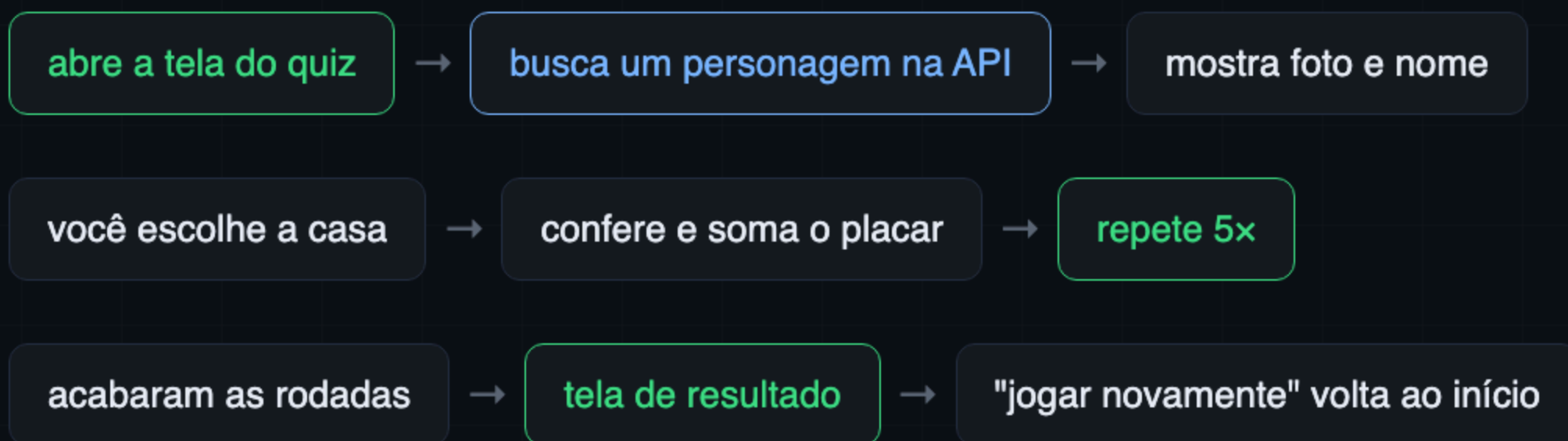
Tela de resultado

↓ "jogar novamente"

volta pro início

— JUNTANDO TUDO

O Hogwarts Quiz, de ponta a ponta



Cada peça que a gente viu nos 2 dias, conectada.

— PARA CONTINUAR

Próximos passos ✨

- ◇ Guardar dados no celular com **Room** e **SQLite**.
- ◇ Tela de **carregando** e aviso de erro de rede.
- ◇ **Animações** entre as rodadas.
- ◇ Fazer o quiz com **outros valores**: apelido, dia que nasceu, ator, etc.

Obrigado! **Bora codar.**

RECURSOS

developer.android.com/courses

developer.android.com/jetpack/compose

kotlinlang.org/docs

github.com/android/nowinandroid